

Definición de Arquitectura de Software

SGPE - Sistema de Gestión de Prácticas Empresariales

María José Realpe Vallejo

Santiago Acosta Calvo

Jeshua Gómez Cortés

Sara Camacho Rodríguez

Samuel Tabares

Corporación Universitaria Empresarial Alexander von Humboldt

Facultad de Ingenierías y Ciencias Básicas

Ingeniería de Software II

Armenia - Quindío

2026

SGPE - Arquitectura actualizada según repositorio

1. Título del proyecto

Nombre del proyecto: SGPE - Sistema de Gestión de Prácticas Empresariales.

Fecha: 01/06/2026.

2. Introducción

El SGPE es una plataforma web orientada a centralizar el proceso de prácticas empresariales de la Corporación Universitaria Empresarial Alexander von Humboldt. La solución permite administrar estudiantes, empresas, vacantes, asignaciones, documentos, seguimientos, evaluaciones y cierre de prácticas desde un mismo sistema.

La arquitectura se ajustó a la estructura real del repositorio, donde el proyecto se organiza en tres bloques principales: backend, frontend y documentación. Esta división facilita el desarrollo colaborativo, separa responsabilidades y permite que cada parte del sistema evolucione sin afectar directamente a las demás.

3. Estructura real del repositorio

El repositorio tiene una separación clara entre la lógica del servidor, la interfaz web y los documentos del proyecto. Esta estructura es coherente con una arquitectura cliente-servidor apoyada por capas internas en el backend.

SGPE - Estructura actual del repositorio usada en la arquitectura



La carpeta backend contiene un proyecto Java con Maven, identificado por el archivo pom.xml y la ruta src/main/java/co/edu/cue/practicass. La carpeta frontend contiene una aplicación web Node con package.json, index.html, public y src. La carpeta docs se reserva para documentación técnica, arquitectura, pruebas y evidencias.

4. Objetivos de la arquitectura

- Separar la interfaz de usuario, la lógica de negocio y la persistencia de datos para mejorar el mantenimiento del sistema.
- Permitir que el frontend consuma el backend mediante servicios o API sin mezclar responsabilidades.
- Organizar el backend en paquetes y capas que faciliten controladores, servicios, repositorios, modelos, seguridad y validaciones.
- Mantener trazabilidad mediante bitácora de auditoría para acciones relevantes del proceso de prácticas.
- Soportar roles y permisos como Administrador, Coordinación Académica, Estudiante, Empresa y Docente Asesor.
- Dejar una base técnica clara para crecimiento futuro en reportes, notificaciones e integraciones.

5. Elección de la arquitectura

El sistema se define como una arquitectura web por capas con separación frontend-backend. En la práctica, el frontend se encarga de la interacción con el usuario, mientras el backend concentra autenticación, autorización, validaciones, reglas de negocio y persistencia.

5.1 Capas propuestas

Capa	Ubicación en el repo	Responsabilidad
Presentación	frontend/src, public, index.html	Pantallas, formularios, navegación, validaciones visuales y consumo de API.
Aplicación / API	backend/src/main/java/co/edu/cue/practicas	Controladores, endpoints, seguridad, manejo de peticiones y respuestas JSON.
Negocio	backend/src/main/java/co/edu/cue/practicas	Reglas de prácticas, estudiantes, empresas, vacantes, seguimiento y evaluación.
Datos	backend/src/main/resources y repositorios	Configuración, conexión con base de datos, consultas y persistencia.
Documentación	docs	Soporte de arquitectura, pruebas, diagramas y entregables del proyecto.

5.2 Patrones de diseño aplicables

Los patrones se mantienen alineados con la lógica del negocio y con la estructura del repositorio. No se agregan responsabilidades nuevas; solo se ordena mejor cómo deberían implementarse.

- Adapter para importar archivos Excel o integrar servicios externos sin acoplar el núcleo del sistema.
- State para manejar el ciclo de vida de vacantes, prácticas, evaluaciones y cierre.
- Observer para notificaciones y alertas derivadas de cambios relevantes.
- Builder para construir instancias de práctica a partir del catálogo y parámetros configurados.

- Chain of Responsibility para validar requisitos académicos, documentales y de acceso.

6. Descripción de componentes

Los componentes principales se relacionan directamente con las carpetas del proyecto. Esta relación ayuda a que la arquitectura no quede como un dibujo separado del código, sino como una guía práctica para implementar el sistema.

6.1 Frontend

El frontend ubicado en la carpeta frontend representa la aplicación web visible para los usuarios. Debe incluir vistas, componentes, formularios, rutas y consumo de servicios del backend. Su responsabilidad es presentar información y capturar datos, no ejecutar reglas de negocio críticas.

6.2 Backend

El backend ubicado en PracticasEmpresariales/backend corresponde al núcleo del sistema. Al estar organizado como proyecto Maven, debe contener paquetes para controladores, servicios, entidades, repositorios, DTO, seguridad, excepciones y validaciones. La lógica de negocio de estudiantes, empresas, vacantes, prácticas, seguimiento y evaluación debe vivir en esta capa.

6.3 Recursos y configuración

La carpeta src/main/resources debe concentrar archivos de configuración, propiedades de conexión, parámetros del sistema y recursos necesarios para ejecutar el backend. Esta capa sirve de apoyo para conectar el sistema con la base de datos y otros servicios.

6.4 Pruebas

La carpeta src/test permite ubicar pruebas unitarias e integrales del backend. Las pruebas deben priorizar login, control de acceso, creación de estudiantes, empresas, vacantes, vinculación y cierre, porque son los procesos esenciales para demostrar el sistema.

6.5 Documentación

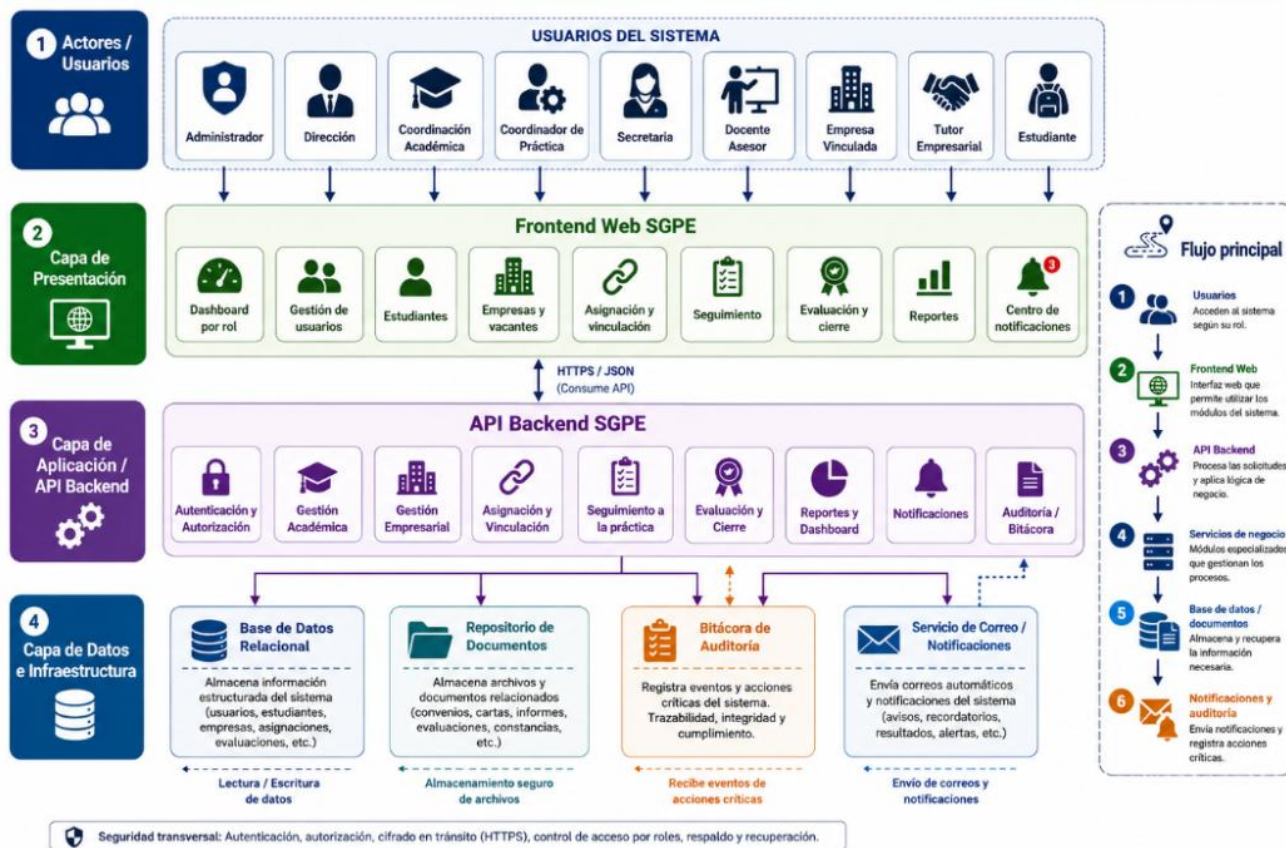
La carpeta docs debe contener los documentos formales del proyecto, incluyendo arquitectura, plan de pruebas, casos de uso, diagramas y evidencias. Esto facilita que el repositorio también sirva como soporte de entrega académica.

7. Diagramas de arquitectura actualizados

7.1 Diagrama de alto nivel

El diagrama de alto nivel muestra cómo los usuarios interactúan con el frontend, cómo el frontend consume el backend y cómo el backend consulta la base de datos. También se incluye la documentación como soporte técnico del proyecto.

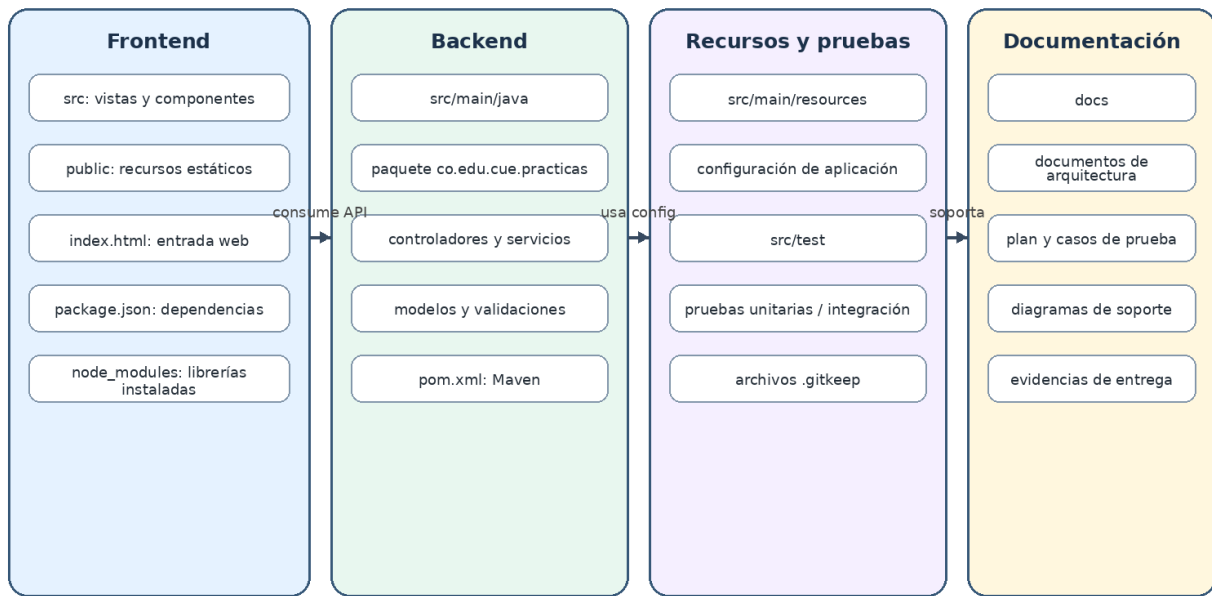
SGPE – Diagrama de Arquitectura de Alto Nivel



7.2 Diagrama de componentes

El diagrama de componentes organiza los bloques del sistema de acuerdo con la estructura real del repositorio. Esto permite explicar qué parte del proyecto corresponde a interfaz, qué parte corresponde a lógica del servidor y qué parte se usa como soporte documental.

SGPE - Diagrama de componentes según carpetas del repositorio

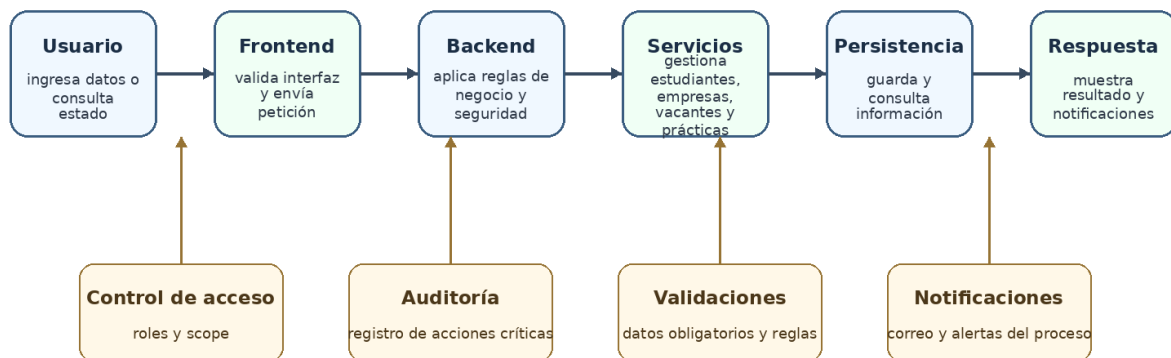


Nota: la lógica de negocio se mantiene en el backend; el frontend solo consume servicios y muestra la información por rol.

7.3 Diagrama de flujo de datos

El flujo de datos conserva la lógica del negocio: el usuario realiza una acción, el frontend envía la solicitud, el backend valida permisos y reglas, los servicios procesan el caso de uso, la persistencia guarda o consulta datos y finalmente el sistema responde al usuario.

SGPE - Flujo de datos frontend-backend



El flujo mantiene la lógica del negocio: control por rol, gestión académica, gestión empresarial, seguimiento, evaluación y cierre de prácticas.

8. Tácticas Arquitectónicas y Estímulos de Calidad

Las tácticas arquitectónicas permiten mejorar la calidad del sistema SGPE sin cambiar su lógica de negocio. Estas tácticas ayudan a que el sistema sea más seguro, organizado, fácil de mantener y estable durante su uso.

8.1 Seguridad

Táctica:

El sistema controla el acceso mediante autenticación, roles y permisos. Cada usuario solo puede ingresar a las funciones que le corresponden según su rol, como administrador, estudiante, empresa, docente asesor o coordinación académica.

Estímulo:

Un usuario intenta acceder a un módulo que no corresponde a su rol.

8.2 Disponibilidad

Táctica:

El sistema debe manejar errores de forma controlada para evitar bloqueos o cierres inesperados. Si ocurre un problema, debe mostrar un mensaje claro al usuario.

Estímulo:

Un usuario intenta guardar información incompleta o inválida.

8.3 Rendimiento

Táctica:

El sistema separa el frontend, el backend y la base de datos para que cada parte cumpla una función específica. Además, los listados deben usar filtros para evitar cargar demasiada información al mismo tiempo.

Estímulo:

Varios usuarios consultan estudiantes, empresas o vacantes al mismo tiempo.

8.4 Mantenibilidad

Táctica:

El proyecto está organizado por carpetas y capas. El backend se encuentra en Java con Maven, el frontend está separado en Node y la documentación se guarda en la carpeta docs.

Estímulo:

El equipo necesita corregir una funcionalidad o agregar una nueva característica.

8.5 Modificabilidad

Táctica:

El sistema está dividido en módulos como estudiantes, empresas, vacantes, seguimiento, evaluación y cierre de práctica. Esto permite modificar una parte sin alterar las demás.

Estímulo:

Se solicita agregar una nueva regla, un nuevo documento o un nuevo estado de práctica.

8.6 Trazabilidad

Táctica:

El sistema registra acciones importantes en una bitácora, como inicios de sesión, cambios de estado, aprobaciones, rechazos o cierres de práctica.

Estímulo:

Un administrador necesita saber quién realizó una acción dentro del sistema.

8.7 Usabilidad

Táctica:

La interfaz debe ser clara y mostrar a cada usuario solo las opciones necesarias según su rol. Los formularios deben tener mensajes sencillos para evitar errores.

Estímulo:

Un usuario registra un estudiante, una empresa o una vacante.

9. Consideraciones finales

El sistema se entiende como una aplicación web separada en frontend, backend y documentación, con un backend Java Maven y un frontend Node. Esta organización permite explicar mejor la implementación y facilita dividir tareas entre los integrantes del equipo.

Los principales retos técnicos son completar el backend, conectar el frontend con los servicios, aplicar correctamente roles y permisos, y mantener la trazabilidad de las acciones críticas. Aun así, la arquitectura seleccionada es adecuada para una entrega académica porque es simple de implementar, fácil de sustentar y coherente con la lógica del negocio del SGPE.

10. Referencias

- Estructura actual del repositorio ProyectoNuclear [practicas-empresariales].
- Documento de arquitectura SGPE previamente elaborado.
- Requisitos funcionales del Sistema de Gestión de Prácticas Empresariales.
- Plan y casos de prueba del SGPE.